DEPARTMENT OF CSE, CS&IT, AI&DS **2400033271 V.Deepthi**

COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L

FULL STACK APPLICATION DEVELOPMENT

Date of the Session: _/_/_ Time of The Session: ___to_____

#SKILL - 5 Spring Autowiring Demo using @Autowired

Prerequisites:

- General Idea on Spring Framework
- Modules of Spring Framework
- Understanding of Autowiring concepts

A college application maintains student information along with their certification details. The Student object contains a Certification object, and instead of creating this Certification object manually, Spring should automatically provide it using the @Autowired annotation. When the required classes are marked with @Component, Spring creates their objects and manages them inside the IoC container. Using @Autowired, the Certification object is supplied to the Student object by Spring without using the new keyword, allowing both objects to be linked through annotation-based configuration.

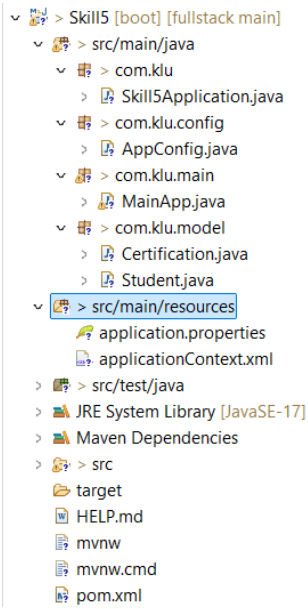
Tasks:

1. Create a Certification class with fields such as id, name, and dateOfCompletion.
2. Create a Student class with fields such as id, name, gender, and a Certification object.
3. Annotate both classes with @Component to enable Spring detection.
4. Use @Autowired on a field, constructor, or setter inside the Student class to inject the Certification object.
5. Configure component scanning using either:

   a. An XML configuration file

   A Java configuration class with @Configuration and @ComponentScan
6. Load the Spring IoC container using ApplicationContext.
7. Retrieve the Student bean and print all details, including the injected Certification object.
8. Push the Spring project to GitHub.

Project Structure:



Skill5Application.java

```
package com.klu;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class Skill5Application {

	public static void main(String[] args) {

		SpringApplication.run(Skill5Application.class, args);

	}

}
```

AppConfig.java

```
package com.klu.config;

import org.springframework.context.annotation.ComponentScan;

import org.springframework.context.annotation.Configuration;

@Configuration

@ComponentScan(basePackages = "com.klu")

public class AppConfig {

}
```

MainApp.java

```
package com.klu.main;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.klu.model.Student;
public class MainApp {
   public static void main(String[] args) {
      ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");
      Student student = context.getBean(Student.class);
      student.displayDetails();
   }
}
```

Certification.java

```
package com.klu.model;
import org.springframework.stereotype.Component;
@Component
public class Certification {
   private int id;
   private String name;
   private String dateOfCompletion;
   public Certification() {
      this.id = 101;
      this.name = "Java Full Stack";
      this.dateOfCompletion = "15-01-2025";
   }
   @Override
   public String toString() {
      return "Certification [id=" + id +
            ", name=" + name +
```

```
        ", dateOfCompletion=" + dateOfCompletion + "]";
    }
}
```

Student.java

```
package com.klu.model;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
@Component
public class Student {
    private int id;
    private String name;
    private String gender;
    @Autowired
    private Certification certification;
    public Student() {
        this.id = 1;
        this.name = "Vasavi";
        this.gender = "Female";
    }
    public void displayDetails() {
        System.out.println("Student ID    : " + id);
        System.out.println("Student Name  : " + name);
        System.out.println("Gender        : " + gender);
        System.out.println("Certification : " + certification);
    }
}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd">
  <!-- Scan SAME base package -->
  <context:component-scan base-package="com.klu"/>
</beans>
```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <parent>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-parent</artifactId>
                <version>4.0.2</version>
                <relativePath/> <!-- lookup parent from repository -->
        </parent>
        <groupId>com.klu</groupId>
        <artifactId>spring-di</artifactId>
        <version>0.0.1-SNAPSHOT</version>
        <name>Skill5-</name>
        <description>Demo project for Spring Boot</description>
        <url/>
        <licenses>
                <license/>
        </licenses>
```

```
<developers>
        <developer/>
  </developers>
  <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
  </scm>
  <properties>
        <java.version>17</java.version>
  </properties>
  <dependencies>
        <dependency>
              <groupId>org.springframework.boot</groupId>
              <artifactId>spring-boot-starter</artifactId>
        </dependency>
        <dependency>
              <groupId>org.springframework.boot</groupId>
              <artifactId>spring-boot-starter-test</artifactId>
              <scope>test</scope>
        </dependency>
  </dependencies>
  <build>
        <plugins>
              <plugin>
                    <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-maven-plugin</artifactId>
              </plugin>
        </plugins>
  </build>     </project>
```

Output:

```
Console ×  Terminal
<terminated> Skill5 - MainApp [Spring Boot App] C:\Users\vasav\Downloads\sts-4.1800.BLEASE\plugins\org.eclipse
Student ID]    :
Student Name   : Deepthi
Gender         : Female
Certification : Certification [id=101, name=Java Full Stack, dateOfCompletion
```

VIVA QUESTIONS:

1. What is autowiring in Spring?
2. What is the purpose of the @Autowired annotation?
3. How does Spring choose which bean to inject?
4. What is component scanning in Spring?
5. What happens if more than one bean matches the dependency type?

1Ans: Autowiring is a feature in Spring where the framework automatically injects dependencies into a bean without explicit configuration.

2Ans: @Autowired tells Spring to automatically inject the required dependency into a constructor, setter, or field.

3Ans: Spring injects a bean based on **type** by default. If multiple beans of the same type exist, it checks for qualifiers or bean names.

4Ans: Component scanning is a mechanism where Spring automatically detects and registers classes annotated with @Component, @Service, @Repository, or @Controller.

5Ans: Spring throws a NoUniqueBeanDefinitionException unless @Qualifier or @Primary is used to specify which bean to inject.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured:_____out of _ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation: